

Azure – Devops Training

Terraform - Variables

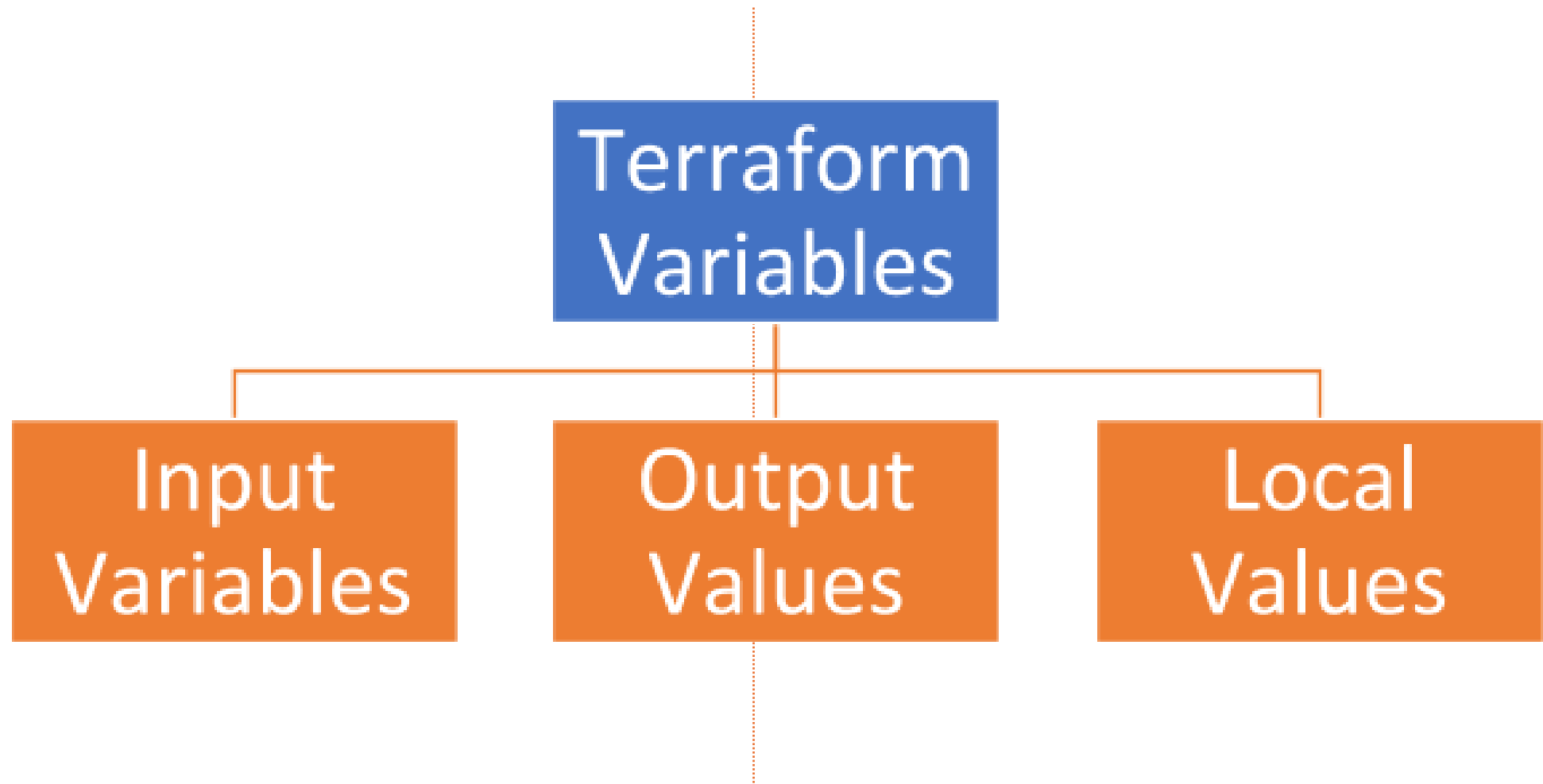
Agenda for the Day

- Terraform Variables
- Terraform Input Variables
- Terraform Output Variables
- Terraform Local Variables

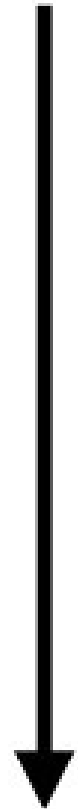
Terraform Variables

Variable a symbolic name associated with a value and whose associated value may be changed

Variables in Terraform are a great way to define centrally controlled reusable values. The information in Terraform variables is saved independently from the deployment plans, which makes the values easy to read and edit from a single file.



Precedence



Command line

Assign with runtime flag such as
`-var='my_var=value'`

`*.auto.tfvars.*`

Dedicated Terraform files just for
assigning values

`terraform.tfvars.json`

Dirk Avery

`terraform.tfvars`

Environment variables

Easy to set, see, change; Use
`TF_VAR_<variable>` pattern

`variables.tf`

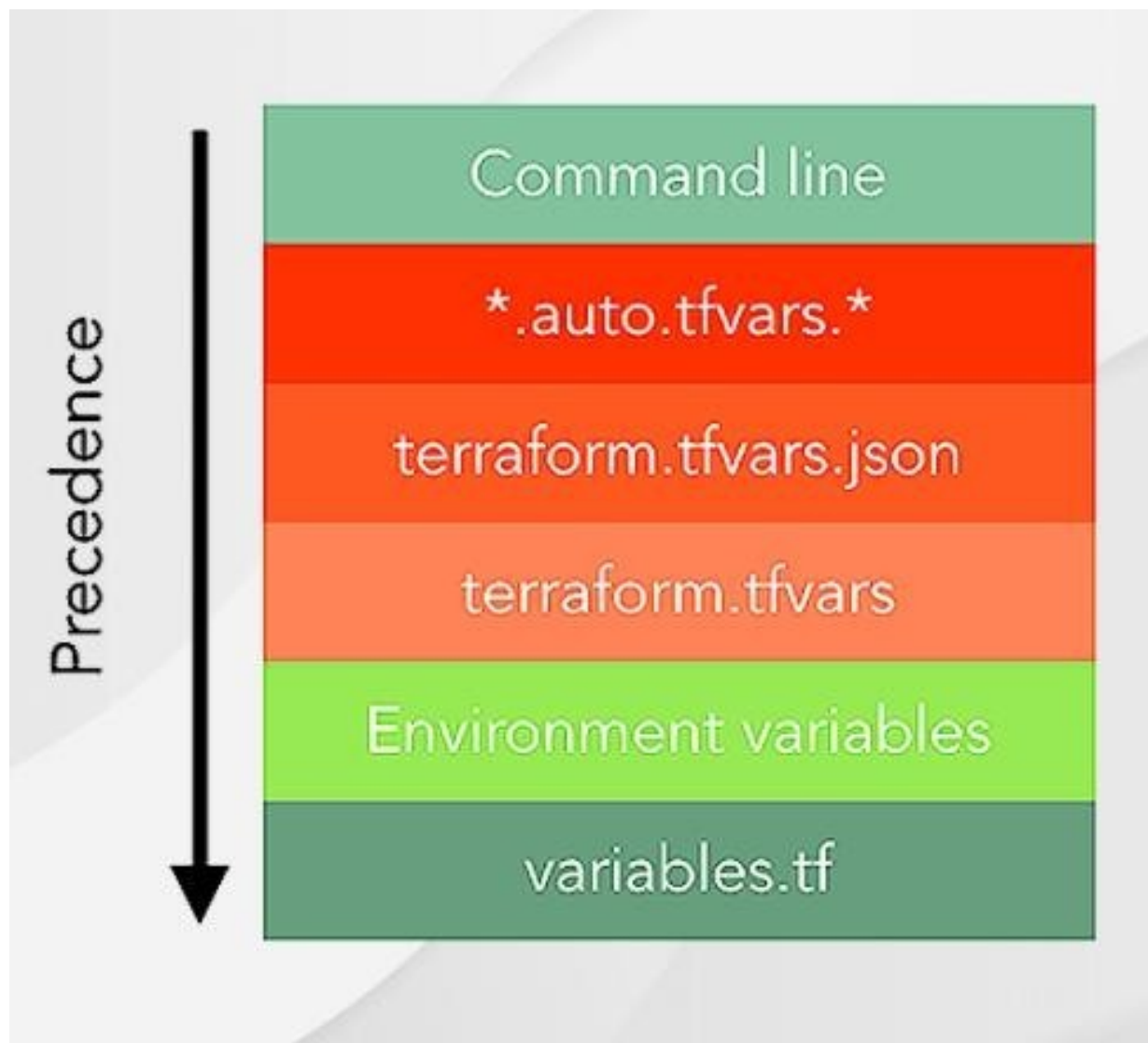
Good for providing non-sensitive
values, reasonable defaults

Interactive

Terraform asks for required values not
otherwise provided

Module invocation

Invoking a module provides
opportunity to assign values



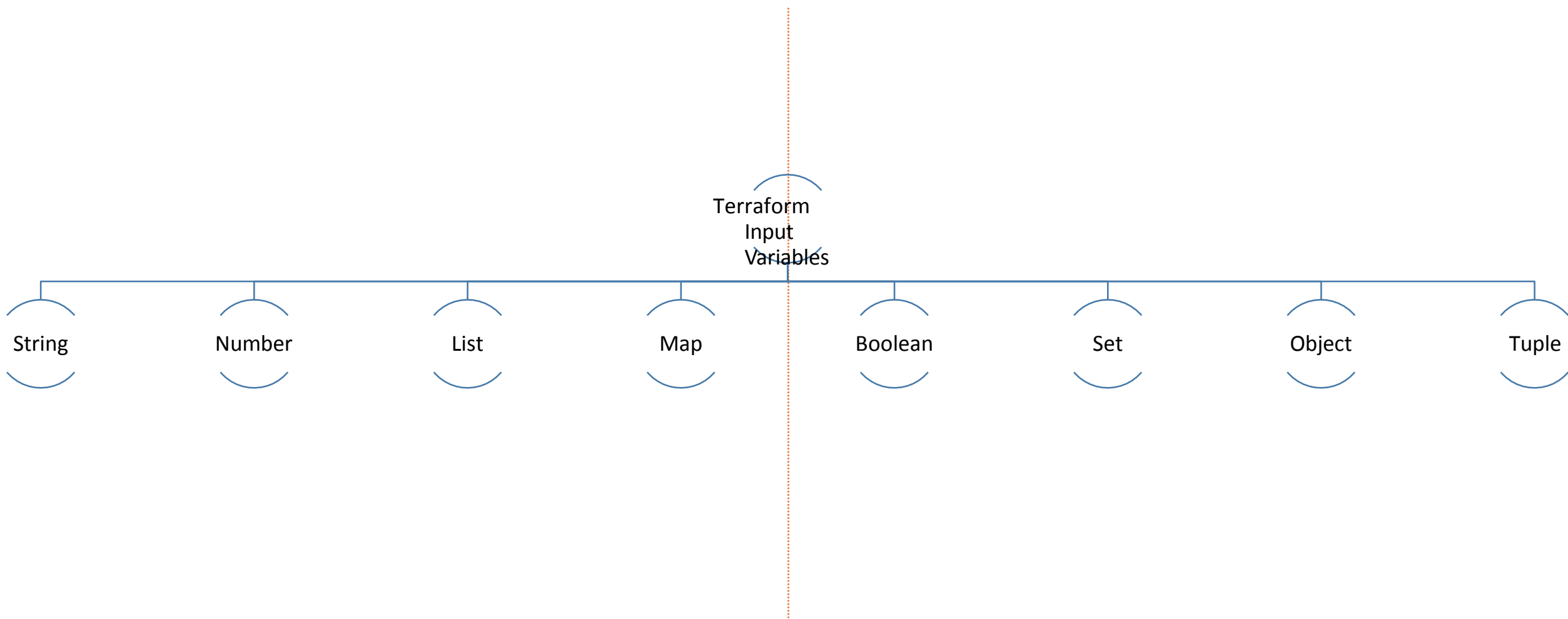
Terraform Input Variables

Terraform Input Variable

Input variables serve as parameters for a Terraform module/Terraform code, allowing aspects of the module/Terraform code to be customized without altering the module's own source code, and allowing modules to be shared between different configurations.

- When you declare variables in the root module of your configuration, you can set their values using CLI options and environment variables.
- When you declare them in child modules, the calling module should pass values in the module block.

Terraform Input Variable Types



Terraform Input Variable

Variable blocks have 5 optional arguments.

We recommend setting a description and type for all variables, and setting a default value when practical.

If you do not set a default value for a variable, you must assign a value before Terraform

can apply the configuration. Terraform does not support unassigned variables.

- `default` - A default value which then makes the variable optional.
- `type` - This argument specifies what value types are accepted for the variable.
- `description` - This specifies the input variable's documentation.
- `validation` - A block to define validation rules, usually in addition to type constraints.
- `sensitive` - Limits Terraform UI output when the variable is used in configuration.

How to declare?

```
variable "vpc_cidr_block" {  
  description = "CIDR block for VPC"  
  type        = string  
  default     = "10.0.0.0/16"  
}
```

Terraform Variable Type: Number

```
1  provider "aws" {  
2      region      = "ap-south-1"  
3      access_key = ""  
4      secret_key = ""  
5  }  
6  
7  variable "numofusers" {  
8      type = number  
9      description = "This is for demo of number variable"  
10     default = 3  
11 }  
12  
13 resource "aws_iam_user" "example" {  
14     count = "${var.numofusers}"  
15     name  = "rajesh.${count.index}"  
16 }
```

Terraform Variable Type: String

```
variable "template" {  
  type = string  
  default = "01000000-0000-4000-8000-000030080200"  
}
```

```
storage = var.template
```

```
1  provider "github" {  
2    token          = ""  
3    organization = "devopsschool-sample-projects"  
4  }  
5  
6  
7  variable "reponame" {  
8    type = string  
9    description = "This is for demo of string variable"  
10   default = "day3-broad"  
11  }  
12  
13  resource "github_repository" "example" {  
14    name          = "${var.reponame}"  
15    description = "My awesome codebase"  
16    private = false  
17  }
```

Terraform Variable Type: List

```
variable "users" {  
    type      = list  
    default = ["root", "user1", "user2"]  
}
```

```
username = var.users[0]
```

```
19 variable "gitrepos" {  
20     type      = "list"  
21     default = ["devopsschool11", "devopsschool2", "devopsschool3"]  
22     description = "This is for demo of list variable"  
23 }  
24  
25 resource "aws_iam_user" "iamuser" {  
26     name = "${var.users[0]}"  
27 }  
28  
29 resource "github_repository" "repo1" {  
30     name = "${var.gitrepos[0]}"  
31     description = "My awesome codebase"  
32     private = false  
33 }  
34  
35 resource "github_repository" "repo2" {  
36     name = "${var.gitrepos[1]}"  
37     description = "My awesome codebase"  
38     private = false  
39 }
```

Terraform Variable Type: Map

```
variable "plans" {  
  type = map  
  default = {  
    "5USD" = "1xCPU-1GB"  
    "10USD" = "1xCPU-2GB"  
    "20USD" = "2xCPU-4GB"  
  }  
}
```

```
plan = var.plans["5USD"]
```

Terraform Variable Type: Map

```
5 variable "amis" {  
6     type = "map"  
7     default = {  
8         "us-east-1" = "ami-b374d5a5"  
9         "us-west-2" = "ami-4b32be2b"  
10    }  
11 }
```

A variable can have a map type assigned explicitly,

Then, replace the `aws_instance` with the following:

```
16 resource "aws_instance" "example" {  
17     ami           = var.amis[var.region]  
18     instance_type = "t2.micro"  
19 }
```

```
54 variable "account_name" {  
55     type = "map"  
56     default = {  
57         "account1" = "devops1"  
58         "account2" = "devops2"  
59         "account3" = "devops3"  
60    }  
61 }  
62 resource "aws_iam_user" "iamuser" {  
63     for_each = var.account_name  
64     name = "${each.value}-iam"  
65 }
```

Terraform Variable Type: Bool

```
variable "set_password" {  
    default = false  
}
```

```
create_password = var.set_password
```

```
terraform apply -var set_password="true"
```

```
1  variable "create_vm" {  
2      description = "If set to true, it will create vm"  
3      type      = bool  
4  }  
5  
6  variable "create_vmss" {  
7      description = "If set to true, it will create vmss"  
8      type      = bool  
9  }  
10  
11  and define the resource azurerm_linux_virtual_machine and azurerm_linux_  
12  
13  resource "azurerm_linux_virtual_machine" "example" {  
14      count      = var.create_vm ? 1 : 0  
15      name       = "example-machine"  
16      resource_group_name = azurerm_resource_group.example.name  
17      location    = azurerm_resource_group.example.location  
18      size        = "Standard_F2"
```

Terraform Output Variables

Terraform Variable: Output

Terraform will store hundreds or even thousands of attribute values for all the defined resources in our infrastructure in state file.

An outputted attributes can not only be used for the user reference but it can also act as an input to other resources being created via Terraform. We can use output variables to organize data to be easily queried and shown back to the Terraform user.

While Terraform stores hundreds or thousands of attribute values for all our resources, we are more likely to be interested in a few values of importance, such as a load balancer IP, VPN address, etc.

Output values are like the return values of a Terraform module, and have several uses:

- A child module can use outputs to expose a subset of its resource attributes to a parent module.
- A root module can use outputs to print certain values in the CLI output after running terraform apply.
- When using remote state, root module outputs can be accessed by other configurations via a terraform_remote_state data source.

Terraform Variable: Output

Declaring an Output Value

Each output value exported by a module must be declared using an `output` block:

```
output "instance_ip_addr" {  
  value = aws_instance.server.private_ip  
}
```

```
output "vpc_id" {  
  description = "ID of project VPC"  
  value       = module.vpc.vpc_id  
}
```

```
output "lb_url" {  
  description = "URL of load balancer"  
  value       = "http://${module.elb_http.this_elb_dns_name}/"  
}  
  
output "web_server_count" {  
  description = "Number of web servers provisioned"  
  value       = length(module.ec2_instances.instance_ids)  
}
```

```
$ terraform output  
lb_url = "http://lb-5YI-project-alpha-dev-2144336064.us-east-1.elb.amazonaws.com/"  
vpc_id = "vpc-004c2d1ba7394b3d6"  
web_server_count = 4
```

Terraform Local Variables

What is local value in terraform?

These are variables that are local to a module. They are defined, assigned, and used in the same module, and defined in the “locals” block. Below is an example snippet on a local block:

Local variables can be declared once and used any number of times in the module. These can be accessed as objects by using the format of "local.Variable_Name".

Unlike variables found in programming languages, Terraform's locals don't change values during or between Terraform runs such as plan, apply, or destroy. You can use locals to give a name to the result of any Terraform expression, and re-use that name throughout your configuration. Unlike input variables, locals are not set directly by users of your configuration.

What is local value in terraform?

Comparing modules to functions in a traditional programming language:

- Input variables are analogous to function arguments and
- Outputs values are analogous to function return values, then
- local values are comparable to a function's local temporary symbols.

When To Use Local Values?

- Local values can be helpful to avoid repeating the same values or expressions multiple times in a configuration, but if overused they can also make a configuration hard to read by future maintainers by hiding the actual values used.
- Use local values only in moderation, in situations where a single value or result is used in many places and that value is likely to be changed in future. The ability to easily change the value in a central place is the key advantage of local values.
- Each locals block can have as many locals as needed, and there can be any number of locals blocks within a module. The names given for the items in the local block must be unique throughout a module. The given value can be any expression that is valid within the current module.
- The expression of a local value can refer to other locals, but as usual reference cycles are not allowed. That is, a local cannot refer to itself or to a variable that refers (directly or indirectly) back to it.



Thank you

